

New code 20th April

```
from ble_keyboard import BLEKeyboard
from machine import Pin, TouchPad
import time

kb = BLEKeyboard("ESP32 Touch RAW")

# Touch pins
touch_a = TouchPad(Pin(4))
touch_s = TouchPad(Pin(12))
touch_d = TouchPad(Pin(13))
touch_f = TouchPad(Pin(14))

# Map: key → (TouchPad, keycode, modifier)
touch_map = [
    ("a", touch_a, 4, 0),
    ("s", touch_s, 22, 0),
    ("d", touch_d, 7, 0),
    ("f", touch_f, 9, 0),
]

THRESHOLD = 180

print("Waiting for connection...")
while not kb.is_connected():
    time.sleep(1)
```

```

print("Connected! Touch to type...")

# Simple debounce tracking
last_pressed = {name: False for name, _, _, _ in touch_map}

while True:
    for name, tp, keycode, modifier in touch_map:
        val = tp.read()

        # Touch detected
        if val < THRESHOLD:
            if not last_pressed[name]:
                print(f"{name} touched → RAW {keycode}")
                kb.send_raw(keycode, modifier)
                last_pressed[name] = True
            else:
                last_pressed[name] = False # reset when released

    time.sleep(0.05)

```

pygame code:

```

import pygame
import sys
import os
import random

```

```
# ————— CONFIG —————
```

```
WIDTH, HEIGHT = 800, 500
```

```
FPS = 60
```

```
SONG_FILE = "36091_Zindagi Do Pal Ki.mp3"
```

```
SONG_VOLUME = 0.4
```

```
CUE_LEAD = 0.3
```

```
STEP = 0.6
```

```
patterns = [
```

```
    (2.0, ["A","S","D","S"], 16),
```

```
    (12.0, ["A","A","S","D"], 16),
```

```
    (22.0, ["A","S","D","F"], 24),
```

```
]
```

```
SEQUENCE = []
```

```
for start, pattern, reps in patterns:
```

```
    for i in range(reps):
```

```
        SEQUENCE.append((start + i * STEP, pattern[i % len(pattern)]))
```

```
FINGER = {
```

```
    "A": ("INDEX", (255,120,120)),
```

```
    "S": ("MIDDLE", (120,200,255)),
```

```
    "D": ("RING", (140,255,180)),
```

```
    "F": ("LITTLE", (255,220,120)),
```

```
}
```

```
# ————— INIT —————
```

```
pygame.mixer.pre_init(44100, -16, 2, 512)
```

```
pygame.init()
```

```
screen = pygame.display.set_mode((WIDTH, HEIGHT))
```

```
clock = pygame.time.Clock()
```

```
font_big = pygame.font.SysFont("consolas", 60, True)
```

```
font_mid = pygame.font.SysFont("consolas", 26)
```

```
font_small = pygame.font.SysFont("consolas", 18)
```

```
def load(p): return pygame.mixer.Sound(p)
```

```
sounds = {
```

```
    "A": load("assets/sound1.wav"),
```

```
    "S": load("assets/sound2.wav"),
```

```
    "D": load("assets/sound3.wav"),
```

```
    "F": load("assets/sound4.wav"),
```

```
}
```

```
if os.path.exists(SONG_FILE):
```

```
    pygame.mixer.music.load(SONG_FILE)
```

```
    pygame.mixer.music.set_volume(SONG_VOLUME)
```

```
# ————— VISUALS —————
```

```
particles = []
```

```
intro_particles = []
```

```
pulse = 0
```

```

def draw_gradient():
    for y in range(HEIGHT):
        pygame.draw.line(screen, (15+y//12,15+y//12,35+y//6),(0,y),(WIDTH,y))

def spawn_particle():
    particles.append([random.randint(0, WIDTH), HEIGHT, random.uniform(0.5,1.5)])

def update_particles():
    for p in particles:
        p[1]-=p[2]
        pygame.draw.circle(screen,(120,140,255),(int(p[0]),int(p[1])),3)
    particles[:] = [p for p in particles if p[1]>0]

def spawn_intro_particle():
    intro_particles.append([
        random.randint(0, WIDTH),
        random.randint(0, HEIGHT),
        random.uniform(-0.3,0.3),
        random.uniform(0.2,0.6),
        random.randint(50,120)
    ])

def update_intro_particles():
    for p in intro_particles:
        p[0]+=p[2]; p[1]-=p[3]; p[4]-=0.3
        if p[4]>0:
            surf=pygame.Surface((WIDTH,HEIGHT),pygame.SRCALPHA)
            pygame.draw.circle(surf,(180,200,255,int(p[4])),(int(p[0]),int(p[1])),3)

```

```

        screen.blit(surf,(0,0))

intro_particles[:] = [p for p in intro_particles if p[4]>0]

def trigger_pulse():

    global pulse

    pulse = 120

def draw_pulse():

    global pulse

    if pulse>0:

        surf=pygame.Surface((WIDTH,HEIGHT),pygame.SRCALPHA)

        pygame.draw.circle(surf,(120,120,255,40),(WIDTH//2,HEIGHT//2),pulse)

        screen.blit(surf,(0,0))

        pulse-=4

def draw_pads(active):

    keys=["A","S","D","F"]

    for i,k in enumerate(keys):

        x=140+i*130; y=360

        if k==active:

            for r in range(3):

                pygame.draw.rect(screen,(140,140,255),

                    (x-5-r*2,y-5-r*2,90+r*4,90+r*4),border_radius=20)

            pygame.draw.rect(screen,(40,40,70),(x,y,80,80),border_radius=20)

            txt=font_mid.render(k,True,(255,255,255))

            screen.blit(txt,(x+25,y+22))

def draw_hud(score, combo):

```

```
pygame.draw.rect(screen,(10,10,20),(0,0,WIDTH,50))

screen.blit(font_small.render(f"Score: {score}",True,(200,200,200)),(20,15))

screen.blit(font_small.render(f"Combo: {combo}",True,(200,200,255)),(WIDTH-
140,15))
```

```
# ————— GAME STATE —————
```

```
playing=False
```

```
cue_index=0
```

```
current=None
```

```
pending=None
```

```
ptime=0
```

```
score=0
```

```
combo=0
```

```
acc_text=""
```

```
acc_timer=0
```

```
intro_alpha=0
```

```
hit_window=0.5
```

```
# ————— MAIN LOOP —————
```

```
running=True
```

```
while running:
```

```
    dt=clock.tick(FPS)
```

```
    for e in pygame.event.get():
```

```
        if e.type==pygame.QUIT:
```

```
running=False
```

```
if e.type==pygame.KEYDOWN:
```

```
    if e.key==pygame.K_ESCAPE:
```

```
        running=False
```

```
    if e.key==pygame.K_SPACE:
```

```
        pygame.mixer.music.stop()
```

```
        pygame.mixer.music.play()
```

```
        playing=True
```

```
        cue_index=0
```

```
        score=0
```

```
        combo=0
```

```
key=pygame.key.name(e.key).upper()
```

```
if key in sounds:
```

```
    sounds[key].play()
```

```
if playing and pending:
```

```
    t=pygame.mixer.music.get_pos()/1000
```

```
    diff=abs(t-ptime)
```

```
if diff<=hit_window:
```

```
    if key==pending:
```

```
        if diff<0.15:
```

```
            res="PERFECT"; score+=2; combo+=1
```

```
        elif diff<0.3:
```

```

        res="GOOD"; score+=1; combo+=1
    else:
        res="MISS"; combo=0
    else:
        res="MISS"; combo=0

    acc_text=res
    acc_timer=0.6
    pending=None

# ————— DRAW —————
draw_gradient()

if not playing:
    if random.random()<0.15:
        spawn_intro_particle()
    update_intro_particles()

    if intro_alpha<255:
        intro_alpha+=2

    l1=font_big.render("we are testing",True,(200,220,255))
    l2=font_big.render("your reflexes",True,(200,220,255))

    l1.set_alpha(intro_alpha)
    l2.set_alpha(intro_alpha)

    screen.blit(l1,(WIDTH//2-l1.get_width()//2,150))

```

```
screen.blit(l2,(WIDTH//2-l2.get_width()//2,210))
```

```
sub=font_mid.render("press SPACE when you're ready",True,(180,180,200))
```

```
sub.set_alpha(intro_alpha)
```

```
screen.blit(sub,(WIDTH//2-sub.get_width()//2,280))
```

```
else:
```

```
    if random.random()<0.25:
```

```
        spawn_particle()
```

```
    update_particles()
```

```
draw_pulse()
```

```
draw_hud(score, combo)
```

```
t=pygame.mixer.music.get_pos()/1000
```

```
if cue_index<len(SEQUENCE):
```

```
    bt,k=SEQUENCE[cue_index]
```

```
    if t>=bt-CUE_LEAD:
```

```
        current=k
```

```
        pending=k
```

```
        ptime=bt
```

```
        cue_index+=1
```

```
        trigger_pulse()
```

```
# AUTO MISS
```

```
if pending and t>ptime+hit_window:
```

```
    acc_text="MISS"
```

```
acc_timer=0.6

combo=0

pending=None

if current:

    finger,col=FINGER[current]

    key_txt=font_big.render(current,True,col)

    screen.blit(key_txt,(WIDTH//2-key_txt.get_width()//2,100))

    finger_txt=font_mid.render(finger,True,col)

    screen.blit(finger_txt,(WIDTH//2-finger_txt.get_width()//2,170))

draw_pads(current)

if acc_timer>0:

    acc_timer-=dt/1000

    y=int(acc_timer*60)

col={"PERFECT":(0,255,0),"GOOD":(255,255,0),"MISS":(255,0,0)}.get(acc_text,(255,255,255))

txt=font_mid.render(acc_text,True,col)

screen.blit(txt,(WIDTH//2-txt.get_width()//2,260-y))

pygame.display.flip()

pygame.quit()

sys.exit()
```